CampiFi — System Architecture & UML Diagrams Guide

Formal Software Engineering Diagrams, Class Relationships, Component Interactions & Network Models

---

This document provides the formal software architecture, UML class models, sequence flows, protocol interactions, and database ER diagrams for **CampiFi**. Each diagram represents key architectural decisions that enable off-grid, high-concurrency campus messaging with zero internet dependency.

1. Multi-Tier Layered System Architecture

CampiFi follows a clean **5-Tier Architecture** designed for strict separation of concerns, maintainability, and security. Each layer only communicates with adjacent layers via defined interfaces and REST/WebSocket protocols.

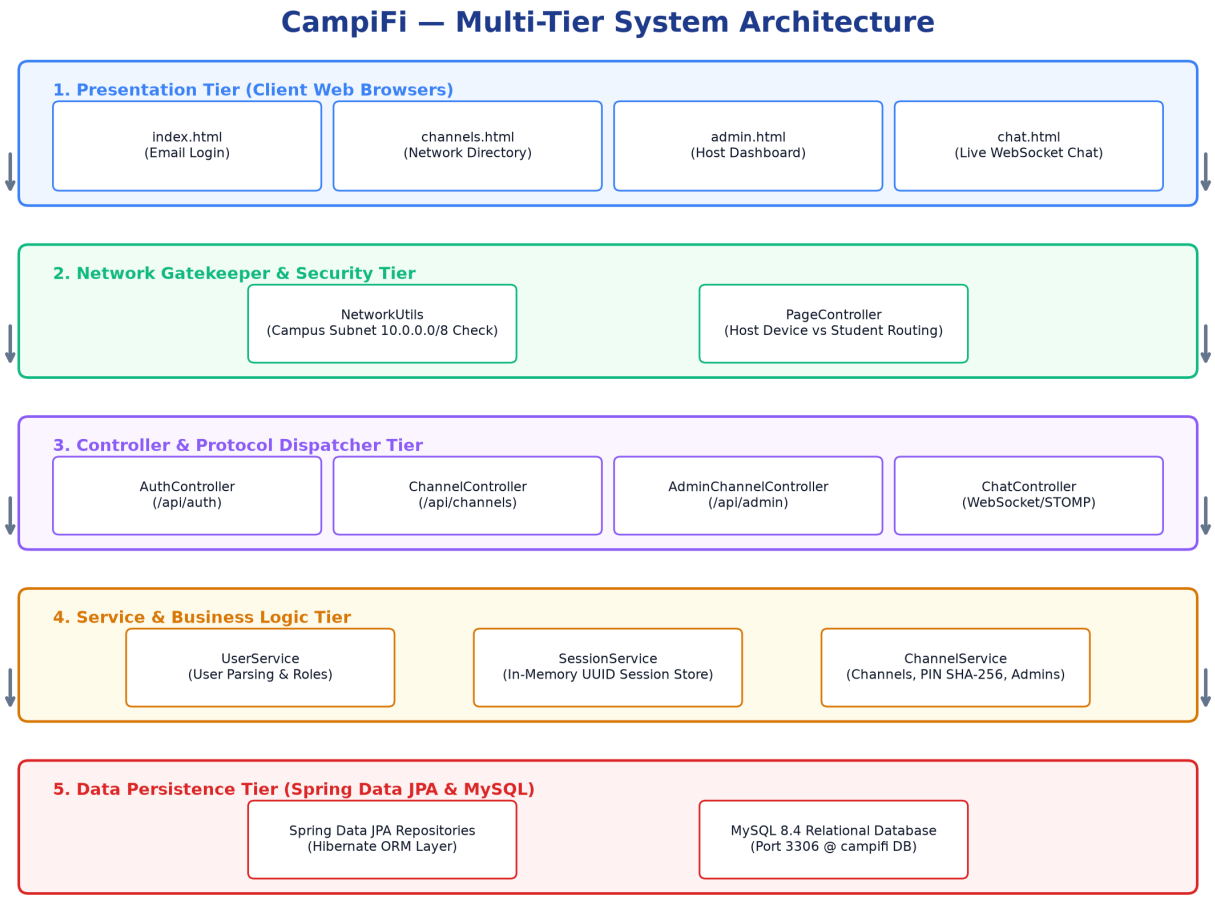


Figure 1: Multi-Tier Layered Architecture of CampiFi

• **Tier 1: Presentation Layer:** Browser-based client interface using HTML5, CSS3, and modern vanilla JavaScript. Connects to backend REST APIs using Fetch/XHR and establishes persistent full-duplex communication using SockJS and STOMP client

libraries.

• **Tier 2: Network Gatekeeper:** Intercepts inbound HTTP connections. Validates client IP against the campus hotspot subnet (10.0.0.0/8) via NetworkUtils. Dynamically routes Host server traffic to admin.html and student devices to index.html via PageController.

• **Tier 3: Controller & Protocol Dispatcher:** Handles incoming HTTP REST endpoints (AuthController, ChannelController, AdminChannelController) and WebSocket frame mappings (ChatController). Validates input payloads and authenticates requests via X-Session-Token headers.

• **Tier 4: Service & Business Logic Layer:** Houses core application logic: user display-name extraction (UserService), in-memory session token lifecycle management (SessionService), SHA-256 PIN cryptography, and dual-tier admin promotion/revocation rules (ChannelService).

• **Tier 5: Data Persistence Layer:** Spring Data JPA with Hibernate ORM translates repository operations into parameterized SQL statements executed via JDBC against a local MySQL 8.4 database running on port 3306.

2. UML Domain Entity Class Diagram

The object-oriented data model uses JPA entity mappings to define tables, attributes, primary/foreign keys, and cardinalities. Instead of unmanaged direct Many-to-Many collections, explicit association entities (ChannelMember and SubChannelAdmin) are used to store administrative flags, ban states, and metadata.

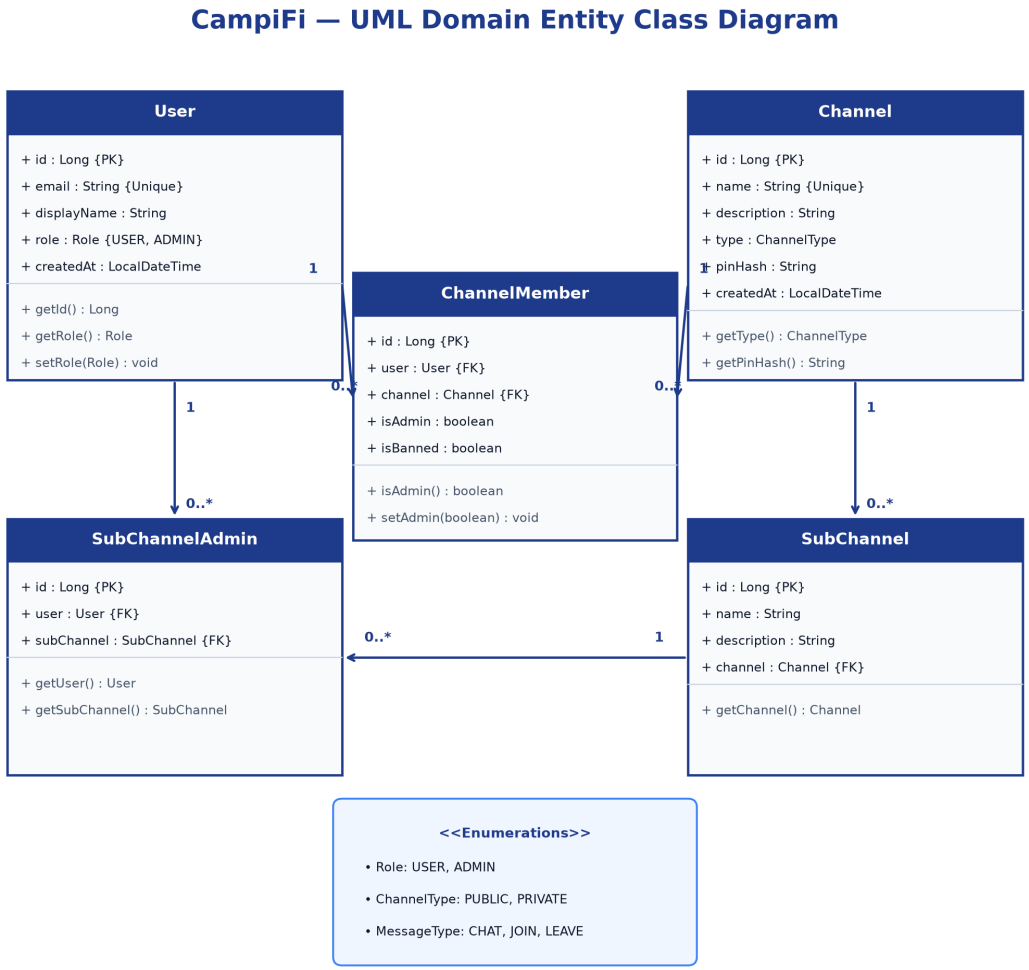


Figure 2: UML Class Diagram Representing Core Domain Entities & Relationships

• **User (Entity):** Represents a university student or faculty member. Identified by unique institutional email. Has a Role (USER or ADMIN). One User can belong to many channels through ChannelMember and administer multiple sub-channels through SubChannelAdmin.

• **Channel (Root Network Entity):** Represents a primary campus network (e.g. *CSE Department*). Can be PUBLIC (open) or PRIVATE (protected by a 4-digit PIN stored as a SHA-256 hash in pinHash). Has a 1-to-Many composition relationship with SubChannel.

• **SubChannel (Entity):** Nested rooms under a parent network (e.g. *Batch A*, *Faculty Lounge*). Contains a @ManyToOne foreign key back to parent Channel. Sub-channels allow granular discussions without polluting root channels.

• **ChannelMember (Association Entity):** Resolves the Many-to-Many relationship between User and Channel. Holds attributes: isAdmin (marks the user as a Master Admin of the channel) and isBanned (restricts access).

• **SubChannelAdmin (Association Entity):** Explicit mapping that assigns a User as an administrator of a specific SubChannel. Enables professors or student leads to moderate individual class groups without granting them master privileges over the entire department.

3. Interaction Sequence Diagram: Private Channel Join Flow

This sequence diagram illustrates the lifecycle of a user attempting to access a PIN-protected private network. It details client-server interaction, cryptographic hashing, database queries, and response dispatch.

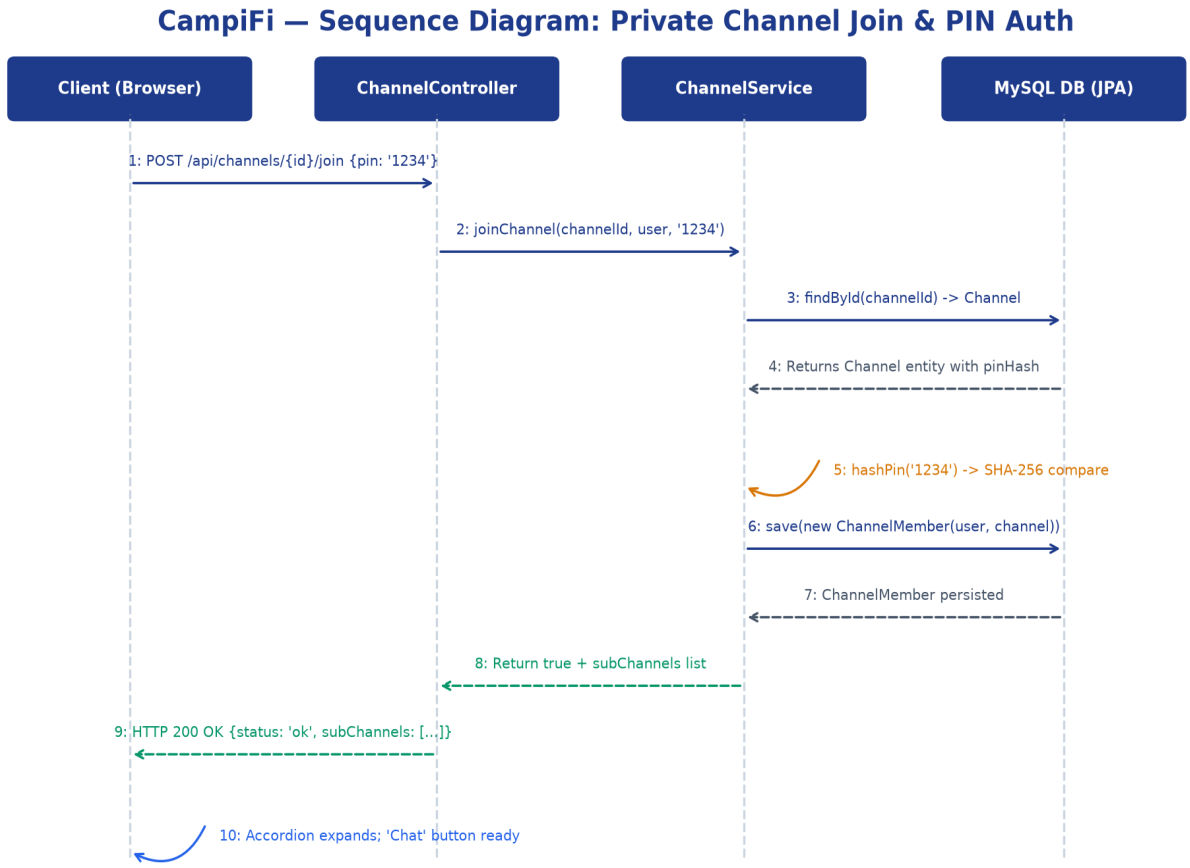


Figure 3: Sequence Diagram for Private Channel PIN Authentication & Join

- **1. User Input:** User clicks a private network on channels.html. A 4-digit PIN prompt appears. Entering the PIN sends an AJAX POST /api/channels/{id}/join with payload { pin: '1234' } and X-Session-Token header.
- **2. Controller Routing:** ChannelController authenticates the session token via SessionService, extracts user details, and invokes ChannelService.joinChannel().
- **3. Database Query:** ChannelService fetches the channel record from ChannelRepository by ID, retrieving the stored pinHash.
- **4. Cryptographic Hashing:** The service computes SHA-256(inputPin) and verifies if the resulting hex string exactly matches channel.getPinHash(). Plaintext PINs are never saved or compared directly.
- **5. Membership Persistence:** Upon a valid hash match and verification that the user is not banned, a new ChannelMember record is persisted in MySQL via ChannelMemberRepository.save().
- **6. Response Dispatch:** The controller returns HTTP 200 OK with the child subChannels array. The frontend expands the channel accordion, enabling the 'Chat' button.

4. Real-Time WebSocket & STOMP Protocol Architecture

Standard HTTP request-response architecture is inefficient for real-time group chat because it requires polling. CampiFi implements full-duplex TCP communication using **WebSockets** with **STOMP** framing and **SockJS** fallback.

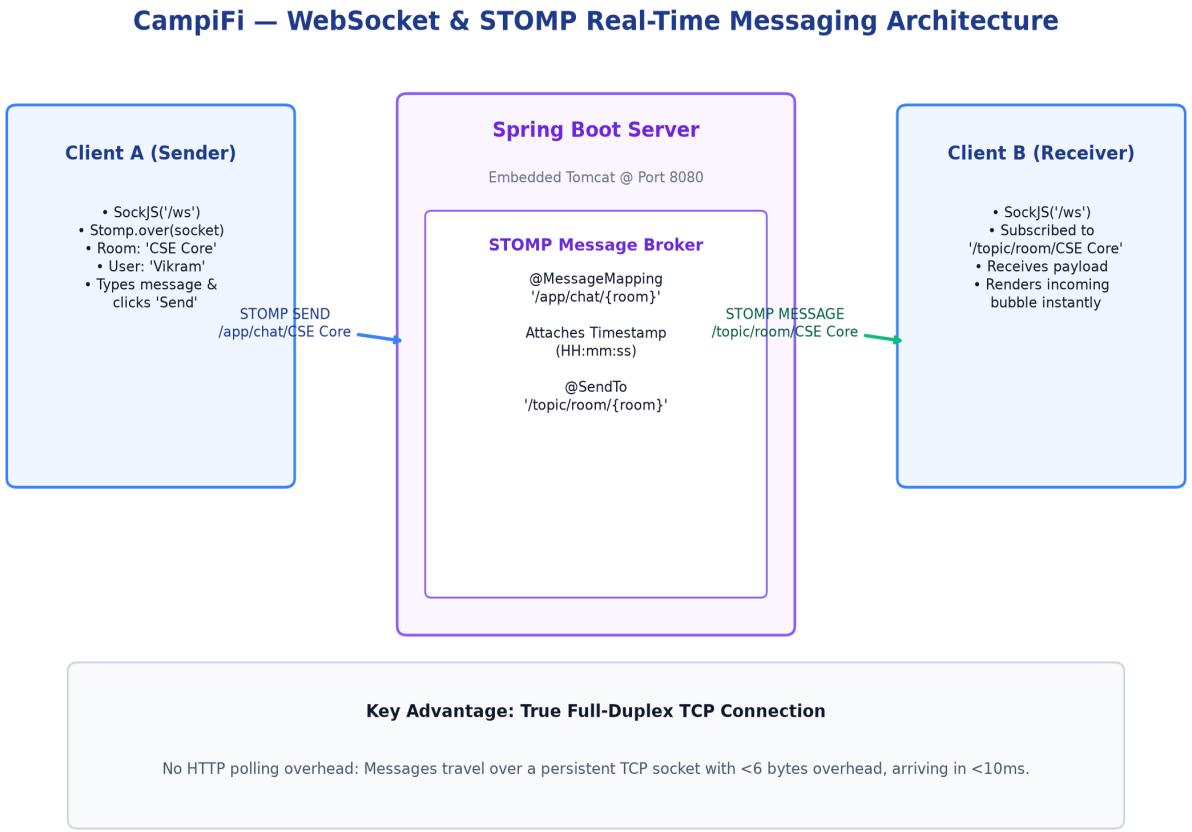


Figure 4: Full-Duplex WebSocket STOMP Publish-Subscribe Messaging Architecture

• **SockJS Handshake (/ws):** The client initiates an HTTP handshake at /ws. If supported, the connection upgrades to a persistent WebSocket TCP stream. If restricted by firewalls, SockJS automatically falls back to HTTP streaming or long-polling.

• **STOMP Over WebSocket:** STOMP adds standard messaging semantics (CONNECT, SUBSCRIBE, SEND). Client A subscribes to /topic/room/CSE Core. Client B subscribes to the same topic.

• **In-Memory Message Broker:** When Client A types a message and clicks Send, a STOMP SEND frame is dispatched to /app/chat/CSE Core. Spring's ChatController receives the payload, assigns a tamper-proof server timestamp, and publishes it to /topic/room/CSE Core.

• **Sub-10ms Latency:** Spring's in-memory broker immediately fans out the message to all active subscribers. The message is rendered inside the chat bubble in real time without refreshing the page.

5. Relational Database Schema & Entity-Relationship (ER) Model

The relational database schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and maintain referential integrity across users, channels, sub-channels, and roles.

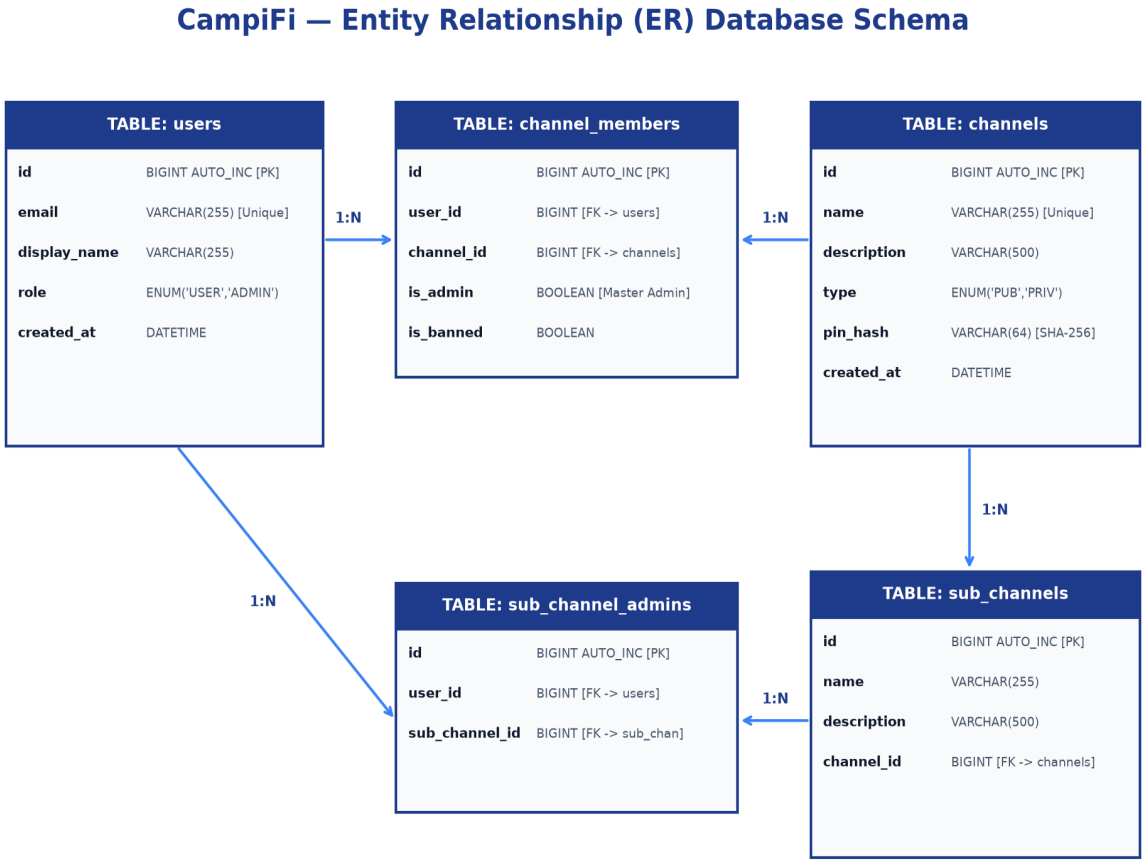


Figure 5: Entity-Relationship (ER) Diagram of the MySQL Database Schema

- **Table 'users':** Stores primary user identity. id is the Auto-Increment Primary Key. email has a UNIQUE constraint. role is stored as an ENUM ('USER', 'ADMIN').
- **Table 'channels':** Stores root networks. name is UNIQUE. type is an ENUM ('PUBLIC', 'PRIVATE'). pin_hash stores the 64-character SHA-256 hash string.
- **Table 'sub_channels':** Represents child channels. channel_id is a Foreign Key referencing channels.id with cascading delete options.
- **Table 'channel_members':** Join table representing channel membership. user_id references users.id; channel_id references channels.id. is_admin designates Master Admins.
- **Table 'sub_channel_admins':** Explicit admin mapping table. user_id references users.id; sub_channel_id references sub_channels.id. Supports clean role delegation and removal.

6. Physical Network Deployment & Campus Hotspot Topology

CampiFi is physically deployed as an autonomous local edge system. The host computer acts as both the wireless router / access point and the application server, requiring no upstream internet gateway.

Component	Network Configuration	Role in CampiFi
Host Machine (Server)	IP: 10.5.4.55 (or 127.0.0.1) Port: 8080 (Tomcat), 3306 (MySQL)	Runs Spring Boot, embedded Tomcat, MySQL server, and the admin.html management dashboard.
Campus Wi-Fi / Hotspot	Subnet: 10.0.0.0/8 (Campus LAN) Broadcast: 10.255.255.255	Provides local Layer 2 / Layer 3 connectivity without requiring WAN/Internet routing.
Student & Faculty Clients	DHCP Range: 10.x.x.x Browser: Chrome, Safari, Firefox	Connects directly to http://10.5.4.55:8080/. Uses index.html, channels.html, and chat.html.